MASTER LIST OF FILENAMES, MODULES, SUBMODULES, CLASSES, FUNCTIONS, SUBROUTINES, EXTERNAL REFERENCES AND METHODS

Revision 0, 6/21/26

This document provides a summary of the various filename, modules, submodules, classes, functions, subroutines, methods and external references associated with the Revision 0 version of the software at the time of uploading to Github.

VB.NET CLIENT PROGRAM

FILENAME: AppConstants.vb

Name	Summary
Module: AppConstants	Defines all application-wide string and numeric constants. Acts as the single authoritative source for pipe names, method names, and timeout values shared across the client's modules.
`PipeName`	Named pipe identifier used by both the client and Fortran server to locate the communication channel. Hard-coded as a deliberate POC simplification; a production system would read this from configuration.
`RequestTimeoutMs`	Global RPC timeout in milliseconds applied to all client-side calls, individual and batch. Set to 5 minutes to accommodate the Mandelbrot benchmark, which can run for several minutes.
`MethodAddInt` ... `MethodDivideInt`	Four constants defining the exact lowercase method names for integer arithmetic operations on the Fortran server. JSON-RPC method matching is case-sensitive, so these must match the server's registration exactly.
`MethodAddReal` ... `MethodDivideReal`	Four constants for the real (floating-point) arithmetic method names registered on the Fortran server.
`MethodAddComplex` ... `MethodDivideComplex`	Four constants for the complex number arithmetic method names.
`MethodSendMessage`	Method name for the echo/send-message round-trip test. The server echoes the string back and the client verifies the response matches.
`MethodNamedParameters`	Method name for the named-parameters demonstration, which sends a JSON object rather than a positional array.
`MethodClientNotify`	Notification method name sent to the server as a fire-and-forget event. The server accepts it silently; no response is expected.
`MethodDisconnect`	Notification method name that signals the server to perform a graceful shutdown. Unique among notifications in that the server acts on it rather than ignoring it.

FILENAME: UIHelperModule.vb

Name	Summary
Module: UIHelperModule	Provides reusable UI utility routines that decouple presentation logic from business logic in Form1. Centralises thread marshaling, status display, message boxes, and text-box helpers.
`RunOnUIThread`	Marshals an action to the UI thread without blocking the caller. Uses `BeginInvoke` rather than `Invoke` to prevent deadlock: async disconnect callbacks arrive on thread-pool threads that may hold StreamJsonRpc internal locks, and synchronous marshaling would stall both threads indefinitely.
`UpdateConnectionStatus`	Updates a label's text and background colour to reflect connected or disconnected state. Delegates to `RunOnUIThread` so it is safe to call from any thread.
`ShowErrorMessage`	Displays a modal error `MessageBox` with a red error icon. Used throughout Form1 to surface validation failures and RPC errors to the user.
`ShowSuccessMessage`	Displays a modal information `MessageBox` confirming a successful operation.
`ShowWarningMessage`	Displays a modal warning `MessageBox` for non-fatal anomalies such as calculation mismatches.
`ClearAndFocusTextBox`	Clears a `TextBox` and moves keyboard focus to it. Used after error conditions to prompt the user to re-enter input.
`ClearTextBox`	Clears a `TextBox` without altering focus. Used by the Clear Log button.

FILENAME: ApplicationEvents.vb

Name	Summary
Class: MyApplication (partial)	Auto-generated VB.NET application event shell. The class body is empty; it exists to allow optional overrides of application-lifecycle events such as startup, shutdown, and unhandled-exception handling without requiring changes to the project template.

FILENAME: CalculationModule.vb

Name	Summary
Module: CalculationModule	Centralises all arithmetic logic and result formatting for the scalar (non-matrix) operations. Keeps calculation, verification, and display concerns out of Form1's event handlers.
`GetOperationInfo`	Maps a server method name (e.g. `"addint"`) to a human-readable display name and operator symbol (e.g. `"Addition"`, `"+"`). Covers all integer, real, and complex methods; used in log messages and result dialogs.
`CalculateExpectedResult`	Performs the same integer arithmetic locally that the Fortran server is asked to perform. Returns the locally expected result so the client can verify the server's response.
`CalculateExpectedRealResult`	Same as above for real (Double) arithmetic. Guards against division by zero by returning `0.0` when the divisor is zero.
`VerifyCalculationResult`	Compares the server's integer result against the locally computed expected value. Returns a named tuple of `(IsMatch, SuccessMessage, ErrorMessage)` with an operation-specific success string.
`VerifyRealCalculationResult`	Compares the server's real result using a hybrid epsilon: relative tolerance (`diff / expected`) for values greater than 1.0 so tolerance scales with magnitude, and absolute tolerance for small values where relative comparison breaks down near zero.
`FormatRealResult`	Formats a `Double` for display. In plain mode uses decimal notation within a readable range; in `forceScientific` mode produces `####.##E##` notation without a leading `+` on positive exponents, matching the Fortran server's output style.
`GetServerMethodName`	Maps the combo-box display string (e.g. `"Add integer"`) back to the server method name (e.g. `"addint"`). Exists as the inverse of the display strings populated in the UI at design time.
`DetermineMethodType`	Classifies a server method name as `"integer"`, `"real"`, `"complex"`, or `"unknown"`. Used by `btnExecute_Click` to route to the correct validation and execution path without a large inline `Select Case`.
`FormatComplexComponent`	Formats a single Double component of a complex number according to a format tag (`"integer"`, `"real"`, or `"scientific"`). Returns an empty string for exactly-zero values to support zero suppression.
`FormatComplexResult`	Assembles a full complex number display string from real and imaginary components. Applies zero suppression (omits a zero component entirely) and sign normalisation (renders `"a - bi"` rather than `"a + -bi"`).
`CalculateExpectedComplexResult`	Computes the locally expected complex result for all four arithmetic operations. Uses standard complex multiplication formula $(ac-bd)+(ad+bc)i$ and division formula with denominator c^2+d^2.
`ParseServerComplexResult`	Parses the complex number string returned by the Fortran server, which may be `"a + bi"`, `"a - bi"`, `"bi"` (pure imaginary), or `"a"` (pure real).

Name	Summary
	Scans right-to-left to correctly handle scientific-notation exponent signs within the string.

FILENAME: ValidationModule.vb (client)

Name	Summary
Module: ValidationModule	Client-specific validation only. Shared validators (non-empty string, integer pair, method selection, stream, error code, echo response, batch result) are provided by `JSONRPCClientLibrary.ValidationModule`; this module holds only the functions unique to the client's input forms.
`ValidateNamedParameters`	Parses a raw user-typed string of JSON key-value pairs (without surrounding braces) into a `Dictionary(Of String, Object)`. Returns the dictionary typed as `Object` so Form1 can pass it directly to the DLL's `SendNamedParametersAsync` without a cast. Normalises `TRUE`/`FALSE` to lowercase before parsing.
`ValidateScientificNotation`	Validates a single real number string and converts it to `Double`. Accepts plain decimal (e.g. `1.5`), integer (e.g. `42`), or scientific notation (e.g. `1.5E03`); requires a mandatory decimal point when an exponent is present. Returns the parsed value and a descriptive error if the format is not recognised.
`ValidateTwoRealParameters`	Calls `ValidateScientificNotation` on both parameter text boxes and returns both parsed `Double` values by reference. Stops at the first failure to give a targeted error message.
`StripAndNormalizeParens`	Private helper that removes optional surrounding parentheses from a complex number input string and trims whitespace. Returns failure if unmatched parentheses are found.
`NormalizeExponent`	Private helper that standardises exponent notation within a numeric token — strips a leading `+` from positive exponents so `Double.TryParse` can handle them uniformly across cultures.
`ClassifyNumericToken`	Private helper that classifies a bare numeric string as `"integer"`, `"real"`, or `"scientific"` based on whether it contains a decimal point or exponent. Used to drive format-promotion logic for complex display.
`ParseComplexComponent`	Private helper that extracts and parses one component (real or imaginary, including its sign) from a pre-split complex token. Returns the numeric value, its format classification, and an error if parsing fails.
`ValidateComplexNumber`	Validates a full complex number string in the form `a+bi`, `a-bi`, `bi`, or `a`. Returns the parsed real and imaginary `Double` values, format classification tags for each component, a canonical display string, and a detailed error message on failure.
`ValidateTwoComplexParameters`	Calls `ValidateComplexNumber` on both parameter text boxes and aggregates the results. Also derives promoted format tags (the "higher" of the two inputs' formats) used for consistent result display.
`PromoteFormat`	Private helper that returns the higher-precedence of two format tags: `"scientific"` beats `"real"` beats `"integer"`. Ensures the result display uses the most expressive format present in either operand.

FILENAME:Form1.vb

Name	Summary
Class: frmMain	The application's single Windows Form. Owns the named pipe and JSON-RPC connection objects and hosts all button event handlers that drive communication with the Fortran server.
`frmMain_Load`	Initialises the form on startup: disables demo control buttons, populates the method combo box, and logs that the client has started.
`InitializeMatrixLabels`	Wires up the 4×4 array of `Label` references (`lblOM`) that map to the original-matrix cells in the TableLayoutPanel on the Matrix tab.
`InitializeModifiedMatrixLabels`	Wires up the corresponding 4×4 `Label` array (`lblMM`) for the result matrix display.
`frmMain_FormClosing`	Ensures the pipe and JSON-RPC session are cleanly closed when the form is closed, preventing the Fortran server from seeing an abrupt connection loss.
`UI`	Shorthand wrapper that calls `UIHelperModule.RunOnUIThread(Me, action)`. Keeps event-handler code readable by avoiding repeated qualification.
`Log`	Appends a timestamped message to the VB log text box and scrolls to the bottom. Marshals to the UI thread so it is safe to call from async callbacks.
`btnConnect_Click`	Establishes the named-pipe connection to the Fortran server. Creates an `RpcTargetHandler`, registers a disconnect callback (which updates status but deliberately does not call `DisconnectPipe` to avoid premature pipe closure), then calls `ConnectionModule.ConnectToServerAsync`.
`SetConnectionStatus`	Delegates to `UIHelperModule.UpdateConnectionStatus` to colour the connection-status label green (connected) or red (disconnected).
`btnSendMessage_Click`	Validates the connection and message text, then sends a `sendmessage` RPC call and verifies that the server echoes the string back unchanged.
`btnSendNotification_Click`	Sends a `clientevent` notification (no response expected) with optional user text appended. Demonstrates the JSON-RPC notification pattern (no `id` field, server does not reply).
`btnExecute_Click`	Central dispatcher for scalar arithmetic. Resolves the selected combo-box item to a server method name, determines whether it is integer, real, or complex, then routes to `ExecuteRealCalculation` or `ExecuteComplexCalculation`, or calls the integer path inline.

Name	Summary
`btnSendNamedParameters_Click`	Validates and parses the named-parameter text box, then sends the parameters as a JSON object (not an array) via `RPCOperations.SendNamedParametersAsync`.
`btnClearLog_Click`	Clears the VB log text box.
`DisconnectPipe`	Tears down the JSON-RPC session and named pipe. Marshals cleanup to the UI thread, nulls all connection fields, updates the status label, and re-enables the Connect button.
`ReconnectToServer`	Performs a full disconnect then reconnect cycle. Used after error-injection tests (-32700/-32600) that forcibly close the connection so further tests can proceed.
`btnStartDemo_Click`	Sends the `mandelbrotbenchmark` RPC call, which runs a long compute job on the Fortran server. Enables/disables demo control buttons and displays elapsed time on completion.
`btnExitDemo_Click`	Sends a `stop` notification to the Fortran server to terminate the benchmark early, then updates button states.
`btnPauseDemo_Click`	Sends a `pause` notification to temporarily suspend the benchmark on the server side without closing the connection.
`btnResumeDemo_Click`	Sends a `resume` notification to restart a paused benchmark.
`btnDisconnect_Click`	Sends a `disconnect` notification to signal graceful server shutdown, waits briefly, then calls `DisconnectPipe` to close the client side.
`btnClose_Click`	Closes the application, which triggers `frmMain_FormClosing` to clean up the connection.
`btnTest32700_Click`	Warns the user that the connection will be lost, then calls the DLL's error-injection function to send a deliberately malformed JSON frame. The server returns a -32700 Parse Error and drops the connection, after which `ReconnectToServer` is called.
`btnTest32600_Click`	Same pattern as the -32700 test but injects an invalid JSON-RPC request (missing required fields), causing a -32600 Invalid Request response.
`btnTestBatchRequest_Click`	Calls `ExecuteBatchRequestAsync` to run three sequential integer operations plus a notification, then formats and displays all results. Logs a per-operation verification for `addint`.
`btnTest32601_Click`	Invokes a method name that does not exist on the server (`nonexistentmethod`). The server returns a -32601 Method Not Found error, which is caught and logged without disconnecting.
`btnTest32602_Click`	Calls `addint` with only one parameter instead of two. The server returns a -32602 Invalid Params error.

Name	Summary
`btnTest32603_Click`	Calls `divideint(10, 0)` to provoke a divide-by-zero on the server. The server returns a -32603 Internal Error.
`GetServerMethodName`	Private passthrough to `CalculationModule.GetServerMethodName`. Keeps the call site in `btnExecute_Click` unqualified.
`ExecuteRealCalculation`	Validates two real parameters, sends the RPC call, formats the result, and verifies it against a locally computed expected value using `VerifyRealCalculationResult`.
`ExecuteComplexCalculation`	Validates two complex parameters, sends the RPC call as a string operand pair, parses the server's string response, reformats using promoted format tags, and verifies both components with `RelativeMatch`.
`RelativeMatch`	Compares two `Double` values using relative tolerance for values greater than 1.0 (so tolerance scales with magnitude) and absolute tolerance for small values to avoid false failures near zero.
`cbxMatrixDataType_SelectedIndexChanged`	When the data-type combo changes, generates fresh random matrix data of the selected type and updates the available operation choices.
`UpdateAvailableOperations`	Rebuilds the operation combo box to show only the operations valid for the currently selected matrix data type (e.g. `transpose` and `inverse` for numeric types but not text).
`btnMatrixExecute_Click`	Validates the connection, validates the three combo-box selections, then routes to the appropriate type-specific execute function (`ExecuteIntegerOperation`, `ExecuteRealOperation`, etc.).
`ExecuteIntegerOperation`	Extracts the integer matrix from the label grid, calls `MatrixIntegerOperationAsync`, validates the response, converts it, and displays the result matrix.
`ExecuteRealOperation`	Same pattern for real (Double) matrices. Handles multiple possible deserialization types (`JArray`, `Double()`, `List(Of Double)`) because the runtime type of `response.matrix` depends on which JSON library deserialized the payload.
`ExecuteTextOperation`	Same pattern for text/character string matrices.
`ExecuteLogicalOperation`	Same pattern for logical (Boolean) matrices. Response values are kept as Fortran-format strings (`.true.` / `.false.`) throughout to preserve the server's native representation.
`ExecuteComplexOperation`	Same pattern for complex number matrices, where each cell is a formatted string such as `"1.5 + 2.3i"`.
`GetSuccessMessageForComplex`	Builds the human-readable success dialog text for a completed complex matrix operation, incorporating the operation name and ordering.

Name	Summary
`GetOperationDisplayName`	Maps an operation code (e.g. `"transpose"`) to a display string (e.g. `"Transpose"`). Used in success messages for integer and real matrix operations.
`GetSuccessMessage`	Builds the success dialog text for integer matrix operations.
`GetSuccessMessageForReal`	Builds the success dialog text for real matrix operations.
`GetSuccessMessageForText`	Builds the success dialog text for text matrix operations.
`GetSuccessMessageForLogical`	Builds the success dialog text for logical matrix operations.
`cbMethods_SelectedIndexChanged`	When the scalar method combo box changes, shows a usage-instructions popup describing the expected input format for that method type (integer, real, complex, or echo).
Class: BatchResult	Private data container holding the per-operation results of one `ExecuteBatchRequestAsync` run: results and error strings for `subtractint`, `multiplyint`, and `addint`, plus a flag indicating whether the notification was sent.
`ExecuteBatchRequestAsync`	Sends three integer RPC calls and one notification sequentially (not in parallel). Sequential sending is required because the Fortran server's named-pipe read loop processes one framed message per cycle and cannot handle concurrent sends that arrive bundled in a single pipe buffer.

FILENAME:MatrixModule.vb

Name	Summary
Module: MatrixModule	All matrix-related logic: data generation, display, validation, extraction, and the five async RPC functions that send matrix operations to the Fortran server. Keeps the substantial matrix-handling code out of Form1.
Class: MatrixOperationResponse	Data-transfer object matching the JSON structure returned by the Fortran server for all matrix operations: `datatype`, `rows`, `columns`, `ordering`, and `matrix` (typed as `Object` because the element type varies by data type).
Class: ValidationResult	Lightweight result container returned by all `Validate*` functions. Holds `IsValid` and `ErrorMessage`; the constructor sets `IsValid = True` with an empty message for the success case.
`InitializeMatrixLabels`	Populates a `Label(,)` array by reading child controls out of a `TableLayoutPanel` at runtime. Called for both the original and result matrix panels to establish the label references used throughout the module.
`GenerateMatrixData`	Dispatcher that fills the original-matrix label grid with random data of the type selected in the combo box. Delegates to one of five private `Generate*` helpers.
`GenerateIntegerMatrix`	Fills the label grid with random integers in the range −99 to 99.
`GenerateRealMatrix`	Fills the label grid with random real values in the range −9.99 to 9.99, formatted to two decimal places.
`GenerateComplexMatrix`	Fills each cell with a random complex number string in the form `a.bb+c.ddi` or `a.bb-c.ddi`.
`GenerateLogicalMatrix`	Fills each cell with `.true.` or `.false.` chosen at random, matching Fortran's boolean literal format.
`GenerateCharacterMatrix`	Fills each cell with a random printable ASCII string via `GenerateRandomCharacterString`.
`GenerateRandomCharacterString`	Generates a random 1-to-4 character string from printable ASCII (codes 33–126). Used to populate text matrix cells with varied content.
`ValidateMatrixIntegerData`	Checks that every cell in the label grid contains a parseable integer. Returns the first cell that fails with a row/column reference in the error message.
`ValidateMatrixRealData`	Checks that every cell contains a parseable `Double`. Accepts plain decimal and scientific notation via `Double.TryParse`.
`ValidateMatrixDimensions`	Confirms the label array has the expected row and column counts. Guards against mismatches if the UI layout changes.
`ValidateComboBoxSelections`	Confirms that the data-type, operation, and ordering combo boxes all have a selection before the execute button proceeds. Returns a specific error message identifying which combo is unset.
`ValidateMatrixNotEmpty`	Ensures every cell in the label grid has non-empty text. Catches cases where `GenerateMatrixData` was never called or a cell was accidentally cleared.

Name	Summary
`ValidateMatrixResponse`	Validates the response object returned by the server: checks that `datatype`, `rows`, `columns`, and `matrix` are all present and that dimensions match expectations.
`ValidateMatrixRealResponse`	Specialised response validation for real matrices. In addition to structural checks, verifies that the `matrix` payload is non-null and contains the expected number of elements.
`ExtractIntegerMatrix`	Reads the label grid in row-major order and returns the values as a flat `Integer()` array ready for transmission.
`ExtractRealMatrix`	Same for real values, returning a `Double()` array.
`DisplayIntegerMatrix`	Writes a flat `Integer()` result array back into the result label grid in row-major order and logs a preview of the first elements.
`DisplayRealMatrix`	Writes a flat `Double()` result array into the result label grid, formatting each value to two decimal places.
`MatrixIntegerOperationAsync`	Sends an integer matrix operation to the Fortran server via `InvokeWithParameterObjectAsync` (named parameters, not positional array). Parses the JSON object response into a `MatrixOperationResponse`.
`MatrixRealOperationAsync`	Same for real matrices. Rounds all values to two decimal places before transmission to avoid floating-point noise in the JSON payload.
`ValidateMatrixTextData`	Validates that every text-matrix cell is non-empty and does not contain control characters or characters that would break the JSON encoding.
`ExtractTextMatrix`	Reads the text label grid into a `String()` array for transmission.
`DisplayTextMatrix`	Writes a `String()` result array into the result label grid.
`MatrixTextOperationAsync`	Sends a text matrix operation with `datatype = "string"`. Text matrices use string arrays throughout, so no numeric conversion is needed.
`ValidateMatrixTextResponse`	Validates the server response for text matrix operations, confirming the matrix payload is a non-null string array with the expected element count.
`ValidateMatrixComplexData`	Validates that every cell in the complex matrix grid contains a well-formed complex number string by calling `ValidateComplexNumberFormat` per cell.
`ValidateComplexNumberFormat`	Private helper that tests a single cell value against the expected complex number pattern. Returns a `ValidationResult` with the cell coordinates in the error message if the format is wrong.
`ValidateMatrixLogicalData`	Validates that every logical-matrix cell contains `.true.` or `.false.` (case-insensitive, Fortran format).
`ExtractLogicalMatrix`	Reads the logical label grid, interpreting `.true.` as `True` and anything else as `False`, returning a `Boolean()` array for transmission.
`DisplayLogicalMatrix`	Writes result data back to the logical label grid. Accepts either `Boolean()` or `String()` (Fortran literals), displaying `.true.`/`.false.` strings directly to preserve Fortran format.

Name	Summary
`MatrixLogicalOperationAsync`	Sends a logical matrix operation. Deserialises the response matrix as `String()` rather than `Boolean()` to preserve the Fortran boolean literal strings returned by the server.
`ValidateMatrixLogicalResponse`	Validates the server response for logical matrix operations.
`ExtractComplexMatrix`	Reads the complex label grid into a `String()` array. Complex values remain as strings throughout (e.g. `"1.5+2.3i"`) because the server expects and returns them in that form.
`DisplayComplexMatrix`	Writes a `String()` complex result array back into the result label grid.
`MatrixComplexOperationAsync`	Sends a complex matrix operation with `datatype = "complex"`. Sends and receives complex numbers as formatted strings, matching the Fortran server's native complex representation.
`ValidateMatrixComplexResponse`	Validates the server response for complex matrix operations.

VB.NET CLIENT PROGRAM - EXTERNAL REFERENCES

Reference	Description
`System.Text` *(Form1.vb, MatrixModule.vb)*	Base .NET namespace for text encoding and string-manipulation types, including `StringBuilder` and the `Encoding` classes. Used throughout for string construction and byte-level text handling in log output and pipe communication.
`System.Threading` *(Form1.vb)*	Provides threading primitives including `CancellationTokenSource` and `CancellationToken`. The client uses `CancellationTokenSource` to cancel in-flight RPC operations and the pipe connection on disconnect.
`System.IO` *(Form1.vb)*	Core file and stream I/O types (`Stream`, `StreamReader`, `StreamWriter`, etc.). Required because `NamedPipeClientStream` inherits from `Stream` and the JSON-RPC layer wraps it as a duplex stream.
`System.IO.Pipes` *(Form1.vb)*	Provides `NamedPipeClientStream` and related pipe types. This is the transport layer the client uses to establish a bidirectional channel to the Fortran server process running on the same machine.
`System.Threading.Tasks` *(Form1.vb)*	Provides `Task` and `Task(Of T)`, the return types for all `Async Function` members. Required explicitly so the compiler resolves `Task` without a full qualification in `Async Function … As Task` signatures.
`System.Runtime.Versioning` *(Form1.vb)*	Supplies the `<SupportedOSPlatform("windows")>` attribute applied to `frmMain`. Named pipes and Windows Forms are Windows-only; the attribute suppresses CA1416 platform-compatibility warnings that the .NET analyser emits for `net8.0-windows` targets.
`System.Windows.Forms` *(UIHelperModule.vb)*	The Windows Forms UI framework — provides `Control`, `Label`, `TextBox`, `ComboBox`, `MessageBox`, `Color`, and all other form and control types used across the application.
`System.Text.Json` *(ValidationModule.vb)*	Microsoft's built-in high-performance JSON library (introduced in .NET Core 3). Used in `ValidateNamedParameters` to parse and validate the user's raw key-value text into a `JsonDocument` before converting it to a `Dictionary(Of String, Object)` for transmission.
`System.Text.RegularExpressions` *(ValidationModule.vb)*	Provides the `Regex` class. Used in `ValidateScientificNotation` to match accepted real-number formats and in `ValidateNamedParameters` to

Reference	Description
	normalise `TRUE`/`FALSE` boolean literals to lowercase before JSON parsing.
`System.Linq` *(Form1.vb, MatrixModule.vb)*	Provides LINQ extension methods, principally `Take()` and `Select()`. Used in log-preview lines to cap the number of matrix elements shown (e.g. `resultMatrix.Take(8).Select(…)`) without modifying the underlying array.
`StreamJsonRpc` *(Form1.vb, MatrixModule.vb)*	NuGet package (v2.24.84) that implements the JSON-RPC 2.0 protocol over any `Stream`. Provides `JsonRpc`, `RemoteInvocationException`, `ConnectionLostException`, and `JsonRpcDisconnectedEventArgs` — the core objects the client uses to make RPC calls, send notifications, and handle disconnection events.
`Newtonsoft.Json` *(MatrixModule.vb)*	Industry-standard JSON serialisation library (Json.NET). Imported at the namespace level in MatrixModule so that `JObject`, `JArray`, and their extension methods are available without repeated qualification when parsing server matrix responses.
`Newtonsoft.Json.Linq` *(Form1.vb, MatrixModule.vb)*	The LINQ-to-JSON sub-namespace of Json.NET. Provides `JObject` and `JArray` directly. Used in both the real-matrix type-conversion block in Form1 (`TypeOf response.matrix Is JArray`) and throughout MatrixModule when deserialising the `matrix` field from the server's JSON response object.
`JSONRPCClientLibrary` *(Form1.vb)*	The companion DLL project in this solution. Importing the namespace brings all public members of its modules (`ConnectionModule`, `RPCOperations`, `ValidationModule`, `AppConstants`, `RpcTargetHandler`, etc.) into scope unqualified, so Form1 can call `ValidateConnection(…)`, `ConnectToServerAsync(…)`, `ExecuteRealOperationAsync(…)` and others without a module prefix.
`Microsoft.VisualBasic.ApplicationServices` *(ApplicationEvents.vb)*	Auto-generated import for the VB.NET application-events infrastructure. Provides the `ApplicationBase` and `WindowsFormsApplicationBase` types that `MyApplication` can override to handle startup, shutdown, and unhandled-exception events. The class body is currently empty; the import is retained by the project template for future use.

VB.NET DYNAMIC LINK LIBRARY PROGRAM

FILENAME: AppConstants.vb

Name	Summary
AppConstants *(Module)*	Defines all operational constants for the DLL. Contains no functions — only named constant values used throughout the other modules.
`PipeName`	The Windows Named Pipe identifier the client uses to connect to the server. Both the client program and the Fortran server must use this same name. Hard-coded as a deliberate POC simplification.
`ReconnectMaxAttempts`	Maximum number of times ReconnectWithRetryAsync will attempt to re-establish the connection before giving up.
`ReconnectDelayMs`	Milliseconds to wait between reconnection attempts. Applied between attempts but not before the first attempt.
`ReconnectPreDelayMs`	Stabilisation pause inserted before reconnecting after a raw error test (-32700, -32600). Allows OS pipe buffers to drain after the server closes the connection.
`RequestTimeoutMs`	Maximum time in milliseconds any single RPC call will wait for a server response before throwing a TimeoutException. Set to 5 minutes to accommodate long-running operations such as the mandelbrot benchmark.
`DisconnectVerificationTimeoutMs`	How long WaitForDisconnectAsync waits for the server to close the connection after a malformed payload is sent. If no disconnect occurs within this window the test is marked unconfirmed.

FILENAME: ClientInfo.vb

Name	Summary
ClientInfo *(Class)*	Simple data class returned by RpcTargetHandler.GetClientInfo() when the server requests information about the connected client. StreamJsonRpc serialises this object to JSON automatically.
`Name`	Human-readable name of the client application.
`Version`	Version string of the client application.
`Platform`	Operating system description, populated at runtime from Environment.OSVersion.
`Runtime`	.NET runtime description, populated at runtime from RuntimeInformation.FrameworkDescription.

FILENAME: ConnectionModule.vb

Name	Summary
ConnectionModule *(Module)*	Manages the complete lifecycle of the named-pipe / JSON-RPC connection: creation, initialisation, cleanup, and reconnection with retry.
`CreatePipeClientAsync`	Creates a NamedPipeClientStream configured for bidirectional asynchronous IO and connects it to the named pipe server. Returns the connected stream for use by InitializeJsonRpc.
`InitializeJsonRpc`	Builds the full StreamJsonRpc transport stack (JsonMessageFormatter + HeaderDelimitedMessageHandler), wires in the RPC target object and disconnect handler, sets SynchronizationContext = Nothing so messages are dispatched on thread-pool threads (preventing UI thread deadlocks), and calls StartListening. Returns the ready-to-use JsonRpc instance.
`ConnectToServerAsync`	Convenience wrapper that calls CreatePipeClientAsync then InitializeJsonRpc and returns all three objects (Rpc, PipeClient, Cts) as a tuple. This is the single entry point callers use to establish a connection.
`CleanupConnection`	Disposes JsonRpc, NamedPipeClientStream, and CancellationTokenSource in the correct tear-down order (RPC first, pipe second, CTS last). Returns True if all three disposed cleanly, False if any step threw an exception.
`ReconnectWithRetryAsync`	Calls CleanupConnection on the existing connection, then attempts to reconnect up to ReconnectMaxAttempts times with ReconnectDelayMs between attempts. Throws InvalidOperationException wrapping the last error if all attempts fail.

FILENAME:RPCOperations.vb

Name	Summary
RPCOperations *(Module)*	Method-neutral wrapper functions for all outbound JSON-RPC 2.0 calls. No server method names are hard-coded; the caller supplies the method name on every call. All functions apply a RequestTimeoutMs deadline and propagate server errors to the caller unchanged.
`WithTimeout(Of T)` *(private)*	Races a Task(Of T) against a Task.Delay timeout. Throws TimeoutException if the delay completes first. Used internally by all value-returning wrapper functions.
`WithTimeout` *(private)*	Void overload of WithTimeout for Task (non-value) operations. Required as a separate overload because Await on a plain Task does not produce a value.
`SendMessageAsync`	Sends a single string parameter to the server and returns the server's string response. Used for echo and message round-trip tests.
`ExecuteOperationAsync`	Sends two Integer parameters to a server method and returns an Integer result. Used for add, subtract, multiply, and divide integer operations.
`ExecuteRealOperationAsync` *(two-param)*	Sends two Double parameters to a server method and returns a Double result. Used for real-number arithmetic operations.
`ExecuteRealOperationAsync` *(one-param)*	Single-parameter overload for server methods that take one Double input, such as the mandelbrot benchmark seed.
`ExecuteStringOperationAsync`	Sends two String parameters and returns a String result. Used when the caller encodes values as strings before sending — for example, complex numbers represented as canonical strings because JSON-RPC 2.0 has no native complex type.
`SendNotificationAsync` *(with message)*	Sends a JSON-RPC 2.0 notification carrying one string message. No response is expected. The id field is correctly omitted from the wire format per the specification.
`SendNotificationAsync` *(no params)*	Parameterless overload for server-control notifications such as "stop", "pause", or "resume" that carry no payload.
`SendNamedParametersAsync`	Sends a request using JSON-RPC 2.0 named (by-name) parameters supplied as a Dictionary(Of String, Object). The server receives a JSON object { "key": value } instead of a positional array.

FILENAME: RpcTargetHandler.vb

Name	Summary
RpcTargetHandler *(Class)*	The server-to-client direction handler. An instance is registered with JsonRpc at startup; StreamJsonRpc dispatches incoming server requests and notifications to matching public methods on this class by name (case-sensitive). Methods that update the UI log must be thread-safe because StreamJsonRpc calls them on background thread-pool threads.
`New`	Constructor. Accepts a logCallback action for writing to the application log and a scrollCallback action for scrolling the log view after notifications. Both callbacks must marshal UI access internally.
`OnServerMessage`	Handles a server request that sends a message string and expects a string acknowledgement. Logs the received message and returns a fixed acknowledgement string.
`OnServerNotification`	Handles a server fire-and-forget notification (no response expected). Logs the notification text and triggers the scroll callback.
`progress`	Receives progress percentage notifications from the server during long-running operations such as the mandelbrot benchmark. Wrapped in Try/Catch so a UI marshalling failure does not crash the background dispatch thread.
`status`	Receives status notifications sent by the server to acknowledge client commands — for example, the "Benchmark PAUSED" or "Benchmark RESUMED" messages sent in response to pause/resume/stop notifications.
`ReceiveData`	Handles a server request that sends a data string and expects a Boolean acknowledgement. Logs the data and returns True.
`GetClientStatus`	Handles a server request for a one-line client status string. Returns a fixed "running and ready" message.
`GetClientInfo`	Handles a server request for structured client information. Builds and returns a ClientInfo object populated with the application name, version, OS, and .NET runtime description.

FILENAME: ValidationModule.vb

Name	Summary
ValidationModule *(Module)*	Centralised validation and utility helpers used by the DLL internally and available to any consuming program. Covers connection state, input parsing, stream readiness, error code checking, echo verification, error data extraction, and batch result verification.
`ValidateConnection`	Checks that a JsonRpc instance is not Nothing and has not been disposed. Returns a clear error message so the caller can inform the user before an ObjectDisposedException surfaces from inside StreamJsonRpc.
`ValidateNonEmptyString`	Returns IsValid=False if the supplied string is null, empty, or whitespace-only. The fieldName parameter is included in the error message for context.
`ValidateIntegerParameter`	Attempts to parse a string as an Integer using Integer.TryParse. Stores the result in a ByRef parameter and returns a descriptive error message on failure.
`ValidateTwoIntegerParameters`	Validates and parses two integer strings in sequence, stopping at the first failure. Delegates to ValidateIntegerParameter for each value.
`ValidateMethodSelection`	Confirms a method name string is not null or empty. The caller must convert any UI combobox selection to a String before passing — UI control objects must not be passed directly.
`ValidateNamedParameters`	Parses a user-supplied key:value string into a typed Dictionary(Of String, Object) by wrapping it in { } and parsing it as JSON. Converts each JSON value to a native .NET type (Integer, Double, String, Boolean, or Nothing).
`ValidateStreamForTest`	Checks that a pipe stream reference is not null and that the stream is currently writable. Used by ErrorTestingModule before writing raw payloads for the -32700 and -32600 tests.
`BuildRawMessage`	Wraps a JSON string in a Content-Length framed message identical to the format used by HeaderDelimitedMessageHandler. Allows malformed JSON to be sent directly to the pipe stream, bypassing StreamJsonRpc's validation — required for the parse error and invalid request tests.
`ValidateErrorCode`	Compares a received RPC error code against an expected value and returns a pass/fail result with a human-readable message. Used by ErrorTestingModule to confirm the server returned the correct standard error code.
`EchoResult` *(Enum)*	Classifies an echo response: ExactMatch, PartialMatch, Mismatch, EmptyResponse, or NullResponse. Returned by ValidateEchoResponse.
`ValidateEchoResponse`	Compares a server's echo response against the original sent message. Returns ExactMatch for identical strings, PartialMatch if the sent message appears within a longer response, and Mismatch otherwise.

Name	Summary
`ExtractErrorData`	Extracts the optional error.data field from a JSON-RPC error response. The parameter is typed as Exception rather than RemoteInvocationException because StreamJsonRpc throws derived types (RemoteMethodNotFoundException, RemoteSerializationException) that are not subclasses of RemoteInvocationException in this library version. Returns HasData=False when no data is present so callers can always check HasData without null guards.
`ValidateBatchAddResult`	Verifies that an AddInt result equals the arithmetic sum of its two inputs. Used after a batch request to confirm the server computed the correct value.

FILENAME: ErrorTestingModule.vb

Name	Summary
ErrorTestingModule *(Module)*	Tests all five standard JSON-RPC 2.0 error codes. Errors -32700 and -32600 are tested by writing malformed payloads directly to the pipe and confirming the server disconnects. Errors -32601, -32602, and -32603 are tested through normal RPC calls that trigger server-side error responses.
`TestResult` *(Class)*	Return type for all test functions. Carries Passed (Boolean), ErrorCode (Integer), Message (String), and NeedsReconnect (Boolean). NeedsReconnect is True for raw-stream tests that destroy the connection.
`SendRawMalformedJson`	Writes a Content-Length framed payload directly to the pipe stream, bypassing StreamJsonRpc entirely. This is the only way to send structurally invalid content — StreamJsonRpc would refuse to serialise it through its normal API.
`WaitForDisconnectAsync` *(private)*	Subscribes a temporary handler to the JsonRpc.Disconnected event and waits up to DisconnectVerificationTimeoutMs for it to fire. Returns True if the server disconnected within the window, False if the timeout elapsed first. Uses TaskCompletionSource for event-driven waiting rather than polling.
`TestRawDisconnectErrorAsync` *(private)*	Shared implementation used by both raw-stream tests. Validates the stream, sends the payload, waits for disconnect, pauses briefly, and reconnects. All error-specific strings and the error code are supplied by the caller so this function stays generic.
`TestParseErrorAsync`	Tests error -32700 by sending JSON with a missing closing brace. Confirms the server rejected the payload by waiting for the Disconnected event, then reconnects automatically.
`TestInvalidRequestAsync`	Tests error -32600 by sending a valid JSON object that is missing the required "jsonrpc" field. Confirms rejection via disconnect and reconnects automatically.
`TestMethodNotFoundAsync`	Tests error -32601 by calling "NonExistentMethod" through the normal RPC path. Catches both RemoteMethodNotFoundException and RemoteInvocationException because StreamJsonRpc may throw either depending on the server's response structure.
`TestInvalidParamsAsync`	Tests error -32602 by calling "AddInt" with one parameter instead of the required two. Both catch blocks handle the same situation because StreamJsonRpc may surface the error as either exception type.
`TestInternalErrorAsync`	Tests error -32603 by calling "DivideInt" with a divisor of zero to provoke a server-side fault. Requires three catch blocks: RemoteMethodNotFoundException (wrong method name), RemoteSerializationException (some servers encode the error code differently in this exception type), and RemoteInvocationException (general fallback).

EXTERNAL REFERENCES

Reference	Description
`System.IO`	Core .NET namespace for stream I/O primitives (`Stream`, `IOException`). Used by `ConnectionModule` and `ErrorTestingModule` to type the pipe stream parameter and catch IO-specific exceptions when writing raw payloads directly to the pipe.
`System.IO.Pipes`	Provides the `NamedPipeClientStream` class and the `PipeDirection` / `PipeOptions` enumerations. Used by `ConnectionModule` to create and connect the named pipe to the Fortran server, and by `ErrorTestingModule` to accept the pipe stream for raw error tests.
`System.Threading`	Provides `CancellationTokenSource` and `CancellationToken`, which are used by `ConnectionModule` to allow connection attempts to be cancelled and to signal cancellation during cleanup.
`System.Threading.Tasks`	Provides `Task`, `Task(Of T)`, and `TaskCompletionSource(Of T)`. Used throughout the DLL to support `Async`/`Await` patterns, `Task.WhenAny`, `Task.Delay`, and the event-driven disconnect wait in `ErrorTestingModule`.
`System.Text.Json`	Microsoft's built-in JSON parsing library. Used by `ValidationModule` to parse user-supplied named-parameter strings (`JsonDocument`, `JsonElement`, `JsonValueKind`) and convert each JSON value to a native .NET type.
`System.Text.RegularExpressions`	Provides the `Regex` class. Used by `ValidationModule` to normalise boolean literals (`TRUE`/`FALSE` in any casing) to lowercase before JSON parsing.
`StreamJsonRpc`	The Microsoft StreamJsonRpc library — the core dependency of the entire DLL. Provides `JsonRpc`, `JsonMessageFormatter`, `HeaderDelimitedMessageHandler`, `JsonRpcDisconnectedEventArgs`, `RemoteInvocationException`, `RemoteMethodNotFoundException`, `RemoteSerializationException`, and `ConnectionLostException`. All RPC communication, message framing, protocol enforcement, and error surfacing go through this library.
`StreamJsonRpc.Protocol`	A sub-namespace of StreamJsonRpc that exposes lower-level protocol types. Imported by `ErrorTestingModule` to access protocol-level constructs needed when inspecting or constructing raw JSON-RPC messages outside of the normal `JsonRpc` API surface.

FORTRAN JSON RPC STATIC LIBRARY PROGRAM REFERENCES

FILENAME: fortran jsonrpc_fortran_lib.f90

Name	Summary
Module: `jsonrpc_types`	Defines the shared data types used across all other modules in the library. Declares the abstract interface for method handlers and the `method_entry` record type used to populate the method registry.
— Abstract interface: `method_handler_interface`	Template signature that every registered RPC method handler must conform to. Requires a json_core context, an inbound params array pointer, an outbound result pointer, and output slots for an error code and error message string.
— Type: `method_entry`	Record that pairs a method name string (up to 64 chars) with a procedure pointer to its handler and a boolean indicating whether the slot is active. Arrays of this type form the method registry.
Module: `jsonrpc_validation`	Provides JSON-RPC 2.0 protocol validation routines, error-code constants, and string-scanning helpers that operate directly on raw JSON text without invoking the json-fortran parser.
— Subroutine: `jsonrpc_skip_whitespace`	Advances a position index past any combination of spaces, tabs, and CR/LF characters within a JSON string. Used by scanning functions to land on the next meaningful character.
— Function: `jsonrpc_scan_to_value`	Locates the value portion of a specific key in a JSON string by searching for the key literal, skipping whitespace, confirming the `:` separator, and returning the position of the value's first character.
— Function: `jsonrpc_is_notification`	Returns `.true.` if the JSON message has no `"id"` field, identifying it as a notification to which the server must not send a reply. Delegates to `jsonrpc_extract_id_token` and checks for the token `null`.
— Function: `jsonrpc_validate_version`	Confirms the `"jsonrpc"` field is present and its value is exactly the string `"2.0"`. Returns `.false.` for any other value or if the field is absent.
— Function: `jsonrpc_is_empty_batch`	Scans past the opening `[` of a batch array and checks whether the next non-whitespace character is `]`, indicating an empty (and therefore invalid) batch.
— Function: `jsonrpc_validate_method_field`	Confirms the `"method"` field is present and that its value begins with `"`, ensuring the method name is a JSON string rather than another type.
— Function: `jsonrpc_get_params_type`	Inspects the first character of the `"params"` value to classify it as an array (`[`), object (`{`), absent, or invalid. Returns one of the four `PARAMS_*` constants.
— Function: `jsonrpc_extract_id_token`	Extracts the raw text of the `"id"` field value (string, integer, or `null` literal) without using the json-fortran parser. Returns the string `'null'` when no `id` field is present.

Name	Summary
— Function: `jsonrpc_validate_id_field`	Verifies the `"id"` field value starts with a character consistent with a valid JSON type (digit, `-`, `"`, or `null`). Returns `.false.` for booleans, arrays, and objects, which are prohibited as ID values.
— Function: `jsonrpc_is_all_notification_batch`	Scans a batch array using a character-level state machine to determine whether every item in the batch omits an `"id"` key. Uses an `escaped` boolean flag to correctly track backslash sequences inside string values. Returns `.true.` only if no item has an `id` field, meaning the server must send no response at all.
— Function: `jsonrpc_build_error_response`	Assembles a complete JSON-RPC 2.0 error response string using string concatenation, without relying on json-fortran. Used on error paths where the json-fortran object tree is unavailable.
— Function: `escape_json_string` *(private)*	Escapes the seven special JSON characters (`"`, `\`, BS, HT, LF, FF, CR) in a string using a two-pass approach: pass 1 counts the exact output length, pass 2 writes the escaped bytes into a precisely allocated output string.
Module: `jsonrpc_utils`	Provides low-level transport utilities for parsing HTTP-style Content-Length framing and generating timestamps. Has no dependencies on json-fortran.
— Function: `GetTimeStamp`	Returns the current local time as an 8-character `HH:MM:SS` string by calling the Fortran intrinsic `date_and_time`.
— Function: `FindHeaderEnd`	Scans a raw byte buffer for the `CR LF CR LF` sequence that marks the end of HTTP-style headers. Returns the position of the last header character, or 0 if not found.
— Function: `ParseContentLength`	Searches the header block for the `Content-Length:` token, extracts the numeric value following it, and returns it as an integer. Returns 0 if the header is absent or the value cannot be parsed.
Module: `jsonrpc_server_state`	Holds the two pieces of per-connection runtime state shared between the transport layer and application handler code: the active pipe handle and a flag indicating whether the current request is a notification.
— Subroutine: `StorePipeHandle`	Writes the supplied Windows named pipe HANDLE into the module-level `g_hPipe` variable, making it accessible to handler code via host association or `GetPipeHandle`.
— Subroutine: `StoreNotificationMode`	Sets the module-level `g_is_notification_mode` flag so that handler subroutines (such as the benchmark) can suppress progress notifications when no reply is expected.
— Function: `GetPipeHandle`	Returns the currently stored pipe handle. Used by handler code that needs to write progress notifications directly to the pipe outside the normal request/response cycle.
— Function: `GetNotificationMode`	Returns the current notification-mode flag. Handlers call this to decide whether to suppress outbound `SendProgressNotification` calls.

Name	Summary
Module: `jsonrpc_protocol`	Core JSON-RPC 2.0 protocol engine. Manages the method registry, parses and dispatches incoming single requests and batch requests, and serialises success and error responses using json-fortran.
— Subroutine: `jsonrpc_init`	Initialises the method registry by zeroing the slot count, marking all entries as unregistered, and explicitly nullifying all procedure pointers to prevent stale-pointer faults on re-initialisation.
— Subroutine: `jsonrpc_shutdown`	Mirrors `jsonrpc_init`: clears and nullifies the registry, and sets the library-initialized flag to `.false.`. Should be called before the server process exits.
— Subroutine: `jsonrpc_register_method`	Adds a method name / handler procedure pointer pair to the next free slot in the registry. Fails silently (via the `success` flag) if the library is uninitialised or the registry is full.
— Subroutine: `jsonrpc_extract_json`	Strips the Content-Length header from a raw pipe read buffer and returns the JSON body, its byte count, and a success flag. Uses `FindHeaderEnd` and `ParseContentLength` from `jsonrpc_utils`.
— Subroutine: `jsonrpc_parse_batch`	Parses a JSON array of request objects using json-fortran, iterates over each item by calling `jsonrpc_parse_message`, and concatenates non-empty responses into a batch response array. Returns an empty string when every item was a notification.
— Subroutine: `jsonrpc_parse_message`	Full request-dispatch pipeline for a single JSON-RPC message: validates the `jsonrpc` version, extracts and validates the `id` (integer only) and `method` fields, looks up the registered handler, invokes it, and builds either a success or error response. Produces no response for notifications.
— Subroutine: `jsonrpc_create_response`	Builds a success response JSON object (`{"jsonrpc":"2.0","result":…,"id":…}`) using json-fortran. Falls back to a manually assembled error response string if json-fortran serialisation fails.
— Subroutine: `jsonrpc_create_error_response`	Builds an error response JSON object (`{"jsonrpc":"2.0","error":{"code":…,"message":…},"id":…}`) using json-fortran. Falls back to string concatenation if serialisation fails, ensuring a well-formed response is always produced.
Module: `jsonrpc_helpers`	Convenience wrappers over json-fortran's `json%get_child` and `json%get` calls for the most common parameter extraction and result-setting patterns, reducing boilerplate in handler implementations.
— Subroutine: `jsonrpc_get_int_param`	Extracts a positional integer parameter from a params array by 1-based index. Sets `success = .false.` if the params pointer is null or the index is out of range.
— Subroutine: `jsonrpc_get_string_param`	Extracts a positional allocatable string parameter from a params array by 1-based index. Returns `.false.` on a null pointer or missing index.

Name	Summary
— Subroutine: `jsonrpc_set_int_result`	Destroys any existing result_value node and creates a new json_integer node named `'result'` with the supplied integer. Used by handlers to set their return value.
— Subroutine: `jsonrpc_set_string_result`	Destroys any existing result_value node and creates a new json_string node named `'result'` with the supplied trimmed string.
— Subroutine: `jsonrpc_get_named_int_param`	Extracts an integer parameter by key name from a params object (named-parameter calling convention). Clears any json-fortran exception on type mismatch and returns `.false.`.
— Subroutine: `jsonrpc_get_named_string_param`	Extracts an allocatable string parameter by key name from a params object. Clears exceptions on failure and returns `.false.`, leaving the output unallocated.

FILENAME: ServerComm.f90

Name	Summary
Module: `ServerComm`	Transport layer for the JSON-RPC server. Owns the read/dispatch/write loop for a Windows named pipe connection, provides progress and error response helpers, and handles Content-Length framing for all outbound messages.
— Subroutine: `SendErrorResponse`	Manually builds and writes a JSON-RPC error response to the pipe without using json-fortran, making it safe to call on parse-error paths before a json-fortran context exists. Accepts `request_id` as a character string (`'null'` or a decimal integer literal) to accommodate callers that fire before the id field has been extracted.
— Subroutine: `SendProgressNotification`	Builds and sends a JSON-RPC 2.0 notification with method name `"progress"` and a params array of `[percent]` or `[percent, "message"]`. Used by long-running handlers to report completion percentage to the client.
— Function: `escape_simple`	Lightweight JSON string escaper for error messages and notification text. Handles the five most common special characters (`"`, `\`, LF, CR, HT) with a single-pass, pre-allocated output buffer sized at 2× the input length. Less general than `escape_json_string` in the library but sufficient for server-generated strings.
— Subroutine: `ProcessMessages`	Main blocking read loop. Reads framed messages from the named pipe, calls `jsonrpc_extract_json` to strip the Content-Length header, classifies each message as batch/request/notification, dispatches to the appropriate `jsonrpc_parse_*` routine, and writes the response back with a new Content-Length header. Exits on broken-pipe or zero-byte read, signalling client disconnection.
— Function: `json_has_id_key`	Scans the raw JSON string for the pattern `"id"` followed by optional whitespace and `:` to determine whether the message is a request (has an id) or a notification (no id). The colon requirement distinguishes the key from the text `"id"` appearing as a value.

Static Library — External References

File	Description
`jsonfortran.lib`	The compiled json-fortran 9.2.0 static library archive (release build). The linker links this into `JSONRPCStaticLibraryRevA.lib` to resolve all json-fortran API calls (`json%initialize`, `json%deserialize`, `json%get`, `json%serialize`, etc.) made in `fortran jsonrpc_fortran_lib.f90`. Without this file the library cannot be linked.
`libjsonfortrand.lib`	The debug-instrumented variant of the json-fortran static library (the trailing `d` denotes debug). Provided alongside `jsonfortran.lib` as an alternative to link against when building a debug-mode executable; the current `.vfproj` references `jsonfortran.lib` by name, making this file a standby copy for projects that prefer the debug variant.
`json_module.mod`	Fortran compiled module interface file for json-fortran's umbrella `json_module`. Required by the compiler when processing any source file that contains `use json_module`; exposes the `json_core` object, `json_value` pointer type, and all json type constants (`json_array`, `json_integer`, `json_real`, `json_string`, etc.).
`json_value_module.mod`	Compiled interface for `json_value_module`, which defines the `json_value` derived type and its associated procedures. The largest of the json-fortran `.mod` files (165 KB) because `json_value` is the central node type for the entire in-memory JSON tree.
`json_file_module.mod`	Compiled interface for json-fortran's file I/O module. Not directly called by the library, but required by the compiler because `json_module` re-exports it; must be present in the include path for the `use json_module` statement to resolve cleanly.
`json_parameters.mod`	Compiled interface for json-fortran's numeric and string kind constants (default integer width, character kind, etc.). Required at compile time to ensure all json-fortran internal data types are consistent across the compilation unit.
`json_string_utilities.mod`	Compiled interface for json-fortran's internal string conversion and Unicode utilities. Not called directly by library code, but transitively required because `json_module` depends on it; must be present alongside the other json-fortran `.mod` files.
`json_kinds.mod`	Compiled interface for json-fortran's primitive kind parameters (`RK` for real, `IK` for integer, `CK` for character, `LK` for logical). Required at compile time by any module that uses json-fortran types to ensure numeric precision is consistent with the pre-built `jsonfortran.lib`.
`ifwin` *(Intel oneAPI HPC Toolkit)*	Intel Fortran compiler module providing declarations for the full Win32 API surface: function signatures for `CreateNamedPipe`, `ReadFile`, `WriteFile`, `FlushFileBuffers`, `ConnectNamedPipe`, `DisconnectNamedPipe`, `CloseHandle`, `GetLastError`, `PeekNamedPipe`, and all related constants (`PIPE_ACCESS_DUPLEX`, `PIPE_WAIT`, `ERROR_BROKEN_PIPE`, `NULL`, etc.). Used by both `fortran jsonrpc_fortran_lib.f90` (`jsonrpc_server_state`) and

File	Description
	`ServerComm.f90`. Supplied automatically by the oneAPI HPC Toolkit installation; no file needs to be copied to the project folder.
`ifwinty` *(Intel oneAPI HPC Toolkit)*	Intel Fortran companion module to `ifwin` that provides Windows API type aliases used in variable declarations: `HANDLE` (64-bit pipe/object handle), `BOOL` (Win32 boolean return), `DWORD` (32-bit unsigned), and `INT_PTR` (pointer-sized integer used with `LOC()` for buffer passing). Required by any declaration of `integer(HANDLE)`, `integer(BOOL)`, etc. in the library source. Also supplied automatically by the oneAPI HPC Toolkit.

FORTRAN JSON RPC SERVER PROGRAM REFERENCES

FILENAME: FortranServerJSONRPC.f90

Name	Summary
program FortranServerJSONRPC	Main entry point of the JSON-RPC 2.0 server. Initialises the json-fortran library, registers 28 methods via a data-driven method_entry array loop, creates a Windows Named Pipe (\\.\pipe\MyTestPipe), waits for the VB.NET client to connect, then hands control to ProcessMessages. On exit, disconnects and closes the pipe and shuts down the library.

FILENAME: Functions.f90 — module Functions (parent module)

Name	Summary
module Functions	Parent module that owns the interfaces for all 28 module subroutine handler procedures implemented across the submodules. Shared constants and utility subroutines defined here are accessible to all submodules via host association.
BASE_RES, BASE_ITER	Grid resolution (2000 points/axis) and maximum iteration count (5000) used as seed multipliers for the Mandelbrot benchmark. Defined once here; referenced in Functions_Benchmark.f90.
CRLF	Two-character CR+LF sequence (char(13)//char(10)) used in LSP-style Content-Length message framing. Defined once here; used by WriteJsonNotification and the main program's pipe header.
DeterminePrimitiveType	Inspects a json_value node and returns a string label ('INTEGER', 'REAL', 'STRING', 'BOOLEAN', 'NULL', 'UNDETERMINED'). Handles the edge case where json-fortran stores whole-number reals as json_real by comparing the value against its truncated integer equivalent.
jsonrpc_get_real_param	Retrieves the REAL(8) value at a positional index from a JSON params array. Accepts both json_integer and json_real nodes, since json-fortran can represent whole-number reals as either type.
jsonrpc_set_real_result	Destroys any existing result_value pointer and creates a fresh json_real node named 'result'. Ensures the result object is always newly allocated rather than appended to a stale pointer.
ValidateMatrixParams	Validates all standard fields of a 4×4 matrix JSON params object: confirms it is a JSON object, verifies datatype, rows=4, columns=4, operation, ordering="row-major", and that the matrix array contains exactly 16 elements. Returns the matrix_array pointer on success.
BuildMatrixResultHeader	Creates the JSON result object and adds the four standard header fields (datatype, rows=4, columns=4, ordering="row-major"). Called by all matrix handlers before appending the output matrix array.
WriteJsonNotification	Assembles an LSP-style Content-Length: N\r\n\r\nJSON framed message into a fixed buffer and writes it to the named pipe via WriteFile and FlushFileBuffers. On write failure, zeros g_hPipe to stop further sends after client disconnection.
SendProgressToClient	Thin wrapper that formats {"jsonrpc":"2.0","method":"progress","params":[N]} and delegates to WriteJsonNotification. Uses positional array params [N] as required by the VB.NET client — distinct from the object format {"percent":N} used by the static library's SendProgressNotification.
SendStatusToClient	Thin wrapper that formats {"jsonrpc":"2.0","method":"status","params":["msg"]} and delegates to WriteJsonNotification. Used by the Mandelbrot benchmark to send PAUSED, RESUMED, and aborted status strings to the client.

FILENAME: Functions_ArithmeticOps.f90 — submodule (Functions) Functions_ArithmeticOps

Name	Summary
submodule Functions_ArithmeticOps	Implements all integer, real, and complex arithmetic JSON-RPC handlers, plus the private helper utilities they share. GetTwoIntParams and PerformArithmeticOp were moved here from Functions.f90 so all arithmetic helpers are co-located.
GetTwoIntParams	Extracts and validates two integer positional parameters from a JSON params array. Returns both values and a success flag; populates error code and message on failure.
PerformArithmeticOp	Performs integer add, subtract, or multiply selected by op_code, using a 64-bit intermediate to detect 32-bit overflow before truncating to the result. Division is excluded because it requires a prior zero-check.
AddInt_handler	JSON-RPC handler for integer addition. Delegates parameter extraction to GetTwoIntParams and computation to PerformArithmeticOp(OP_ADD).
SubtractInt_handler	JSON-RPC handler for integer subtraction. Delegates to GetTwoIntParams and PerformArithmeticOp(OP_SUBTRACT).
MultiplyInt_handler	JSON-RPC handler for integer multiplication. Delegates to GetTwoIntParams and PerformArithmeticOp(OP_MULTIPLY).
DivideInt_handler	JSON-RPC handler for integer division. Performs an explicit zero-check before dividing; returns JSONRPC_DIVISION_BY_ZERO if the divisor is zero. Handled separately from PerformArithmeticOp because of this pre-condition.
GetTwoRealParams	Extracts and validates two REAL(8) positional parameters from a JSON params array, using jsonrpc_get_real_param which accepts both integer and real JSON nodes.
PerformRealArithmeticOp	Performs real add, subtract, or multiply selected by op_code, rounding the result to 2 decimal places via anint to avoid floating-point display artefacts. No overflow check needed — REAL(8) range is sufficient.
AddReal_handler	JSON-RPC handler for real addition. Delegates to GetTwoRealParams and PerformRealArithmeticOp(OP_ADD).
SubtractReal_handler	JSON-RPC handler for real subtraction. Delegates to GetTwoRealParams and PerformRealArithmeticOp(OP_SUBTRACT).
MultiplyReal_handler	JSON-RPC handler for real multiplication. Delegates to GetTwoRealParams and PerformRealArithmeticOp(OP_MULTIPLY).
DivideReal_handler	JSON-RPC handler for real division with a zero-check. Divides and rounds to 2 decimal places; returns JSONRPC_DIVISION_BY_ZERO if the divisor is zero. Handled separately from PerformRealArithmeticOp because of this pre-condition.

Name	Summary
FormatComplexComponent	Formats a single REAL(8) value as a compact string component. Uses scientific notation for `
ParseComplexString	Parses a complex number string in the format "re±imi" (no brackets) into separate REAL(8) real and imaginary parts. Strips spaces, removes trailing i, and scans for the separator + or - not preceded by an exponent E.
GetTwoComplexStringParams	Extracts two complex-number strings from positional params positions 1 and 2. Returns both strings and a success flag.
BuildComplexResultString	Assembles real and imaginary components into the result string format "re+imi" or "re-imi" using FormatComplexComponent for each part. No brackets; lowercase i; no spaces.
PerformComplexArithmeticOp	Performs complex add, subtract, or multiply selected by op_code on decomposed real/imaginary component pairs. Division is excluded because the denominator zero-check (re_b²+im_b²=0) cannot be expressed as a generic op-code path.
AddComplex_handler	JSON-RPC handler for complex addition. Parses two complex strings, delegates to PerformComplexArithmeticOp(OP_ADD), and formats the result string.
SubtractComplex_handler	JSON-RPC handler for complex subtraction. Parses two complex strings, delegates to PerformComplexArithmeticOp(OP_SUBTRACT), and formats the result string.
MultiplyComplex_handler	JSON-RPC handler for complex multiplication. Parses two complex strings, delegates to PerformComplexArithmeticOp(OP_MULTIPLY), and formats the result string.
DivideComplex_handler	JSON-RPC handler for complex division using the conjugate-of-denominator formula. Computes denom = re_b²+im_b²; returns JSONRPC_DIVISION_BY_ZERO if denom=0. Handled separately from PerformComplexArithmeticOp because of this pre-condition.

FILENAME: Functions_Benchmark.f90 — submodule (Functions) Functions_Benchmark

Name	Summary
submodule Functions_Benchmark	Implements the Mandelbrot CPU benchmark handler and its supporting private routines. Provides signal detection (stop/pause/resume) and progress/status notifications to the client during the long-running computation.
STOP_CHECK_INTERVAL	Private constant (50000). Controls how many grid points are processed between signal polls — balances responsiveness against the overhead of PeekNamedPipe calls.
CHECKSUM_ABORTED	Private constant (−1.0d0). Sentinel value returned by mandelbrot_benchmark when the client sends a stop signal mid-computation; detected by MandelbrotBenchmark_handler to set the appropriate error response.
SIGNAL_NONE/STOP/PAUSE/RESUME	Integer codes (0–3) returned by check_for_signals to communicate which client notification was received during the benchmark loop.
MandelbrotBenchmark_handler	JSON-RPC handler that validates the seed parameter (1–10), prints benchmark configuration, calls mandelbrot_benchmark, detects the aborted sentinel, and returns the checksum as the JSON-RPC result.
mandelbrot_benchmark	Computes a Mandelbrot set checksum over a (BASE_RES×seed)² grid with up to BASE_ITER×seed iterations per point. Uses smooth (continuous) escape-time coloring, accumulates a histogram-weighted checksum, sends 10% progress milestones via SendProgressToClient, and checks for stop/pause signals every 50000 points.
pause_loop	Entered when a PAUSE signal is detected mid-computation. Sends a PAUSED status to the client, polls check_for_signals every 100 ms via SleepQQ until a RESUME notification arrives, then sends a RESUMED status and returns.
check_for_signals	Non-blocking named pipe peek using PeekNamedPipe. Scans available bytes for "method":"stop", "method":"pause", or "method":"resume" substrings; consumes the message with ReadFile if recognised; leaves unrecognised content on the pipe.

FILENAME: Functions_StringOps.f90 — submodule (Functions) Functions_StringOps

Name	Summary
submodule Functions_StringOps	Implements the two string/message operation handlers: simple echo and named-parameter demonstration.
SendMessage_handler	JSON-RPC handler that echoes back the first string parameter. Validates that params is a non-empty array and that param1 is a non-empty string, then returns it as the result.
NamedParameters_handler	JSON-RPC handler that iterates over a JSON object's named fields, prints each key-value-type triple to the console using DeterminePrimitiveType, and returns a string indicating the total count of named parameters received.

FILENAME: Functions_MatrixIntegerOps.f90 — submodule (Functions) Functions_MatrixIntegerOps

Name	Summary
submodule Functions_MatrixIntegerOps	Implements integer 4×4 matrix operation handlers. All three handlers use ValidateMatrixParams and BuildMatrixResultHeader from the parent module.
MatrixIntegerTranspose_handler	Reads 16 integers from the JSON params matrix array into a Fortran (4,4) column-major array (row by row), applies the intrinsic TRANSPOSE(), then writes the result back in row-major order for the client.
MatrixIntegerCopy_handler	Reads 16 integers from the JSON params matrix array into a flat array and writes them back unchanged. Functions as an identity/pass-through operation to verify round-trip correctness.
MatrixIntegerSquare_handler	Reads 16 integers, validates each is within [-100, +100] (preventing 32-bit overflow since 100^2=10000), squares each element, and returns the result array.

FILENAME: Functions_MatrixRealOps.f90 — submodule (Functions) Functions_MatrixRealOps

Name	Summary
submodule Functions_MatrixRealOps	Implements real (REAL(8)) 4×4 matrix operation handlers. All three handlers use ValidateMatrixParams and BuildMatrixResultHeader from the parent module.
MatrixRealCopy_handler	Reads 16 REAL(8) values, validates each is within [-100.0, +100.0] (matching the VB.NET client's signed spinner range), and writes them back unchanged.
MatrixRealTranspose_handler	Reads 16 REAL(8) values with range validation [0.0, +100.0] (matching the VB.NET client's non-negative spinner for transpose), applies TRANSPOSE(), and returns the result in row-major order.
MatrixRealSquare_handler	Reads 16 REAL(8) values with no range validation (REAL(8) has sufficient range), squares each element, and returns the result array.

FILENAME: Functions_MatrixTextOps.f90 — submodule (Functions) Functions_MatrixTextOps

Name	Summary
submodule Functions_MatrixTextOps	Implements text/string 4×4 matrix operation handlers. String values are stored in character(len=6) fixed-length fields.
MatrixTextCopy_handler	Reads 16 string elements from the JSON params matrix array using allocatable string intermediates and a block construct, then writes them back unchanged as a copy operation.
MatrixTextTranspose_handler	Reads 16 strings into a (4,4) Fortran array, applies TRANSPOSE(), and writes the result back in row-major order. Uses a block construct for scoped allocatable-to-fixed-length string assignment.

FILENAME: Functions_MatrixLogicalOps.F90 — submodule (Functions) Functions_MatrixLogicalOps

Name	Summary
submodule Functions_MatrixLogicalOps	Implements logical/boolean 4×4 matrix operation handlers. json-fortran automatically converts VB.NET uppercase True/False to Fortran .TRUE./.FALSE.; output is written as lowercase "true"/"false" strings.
MatrixLogicalCopy_handler	Reads 16 logical values from the JSON params matrix array and writes them back unchanged as "true" or "false" string elements.
MatrixLogicalTranspose_handler	Reads 16 logical values into a (4,4) Fortran array, applies TRANSPOSE(), converts back to row-major order, and outputs each as a "true" or "false" string.

FILENAME: Functions_MatrixComplexOps.F90 — submodule (Functions) Functions_MatrixComplexOps

Name	Summary
submodule Functions_MatrixComplexOps	Implements complex-number 4×4 matrix operation handlers. Complex values are exchanged as bracketed strings "(re±imi)" and converted to/from COMPLEX(8) using private parsing and formatting helpers.
MatrixComplexCopy_handler	Parses 16 complex strings via parse_complex_string into COMPLEX(8) values and writes them back formatted via format_complex_string, functioning as a round-trip identity operation.
MatrixComplexTranspose_handler	Parses 16 complex strings into a (4,4) COMPLEX(8) array, applies TRANSPOSE(), and writes the transposed result back as formatted strings in row-major order.
MatrixComplexSquare_handler	Parses 16 complex strings into COMPLEX(8) values, squares each using Fortran complex arithmetic (c*c), and returns the results as formatted strings. No range validation needed — Fortran handles complex multiplication directly.
parse_complex_string (private)	Parses a "(re±imi)" bracketed complex string by stripping parentheses and trailing i, then scanning from character 2 for a + or - not preceded by an exponent E to locate the real/imaginary split point.
format_complex_string (private)	Formats a COMPLEX(8) value as "(re+imi)" or "(re-imi)" with 2 decimal places using the F0.2 format specifier. Uses a simpler fixed-decimal format compared to BuildComplexResultString in the arithmetic submodule, which uses scientific notation for large/small values.

FILENAME: ConsoleUtils.f90 — module ConsoleUtils

Name	Summary
module ConsoleUtils	Provides Windows console window positioning and titling utilities used at server startup. Depends on ifwin and ifwinty for Win32 API types.
AlignConsoleWindowRight	Retrieves primary screen dimensions via GetSystemMetrics, gets the current console window rectangle via GetWindowRect, and repositions it flush against the right edge of the screen, vertically centred, using SetWindowPos.
SetServerConsoleTitle	Sets the console window title bar text by appending a null terminator and calling SetConsoleTitle. Makes the server window identifiable in the Windows taskbar.

FILENAME: JsonRpcErrorCodes.f90 — module JsonRpcErrorCodes

Name	Summary
module JsonRpcErrorCodes	Defines all integer parameter constants used across the server for error codes and arithmetic operation selection. No subroutines or functions — constants only.
JSONRPC_PARSE_ERROR (−32700)	Standard JSON-RPC 2.0 parse error code, returned when the incoming message cannot be parsed as valid JSON.
JSONRPC_INVALID_REQUEST (−32600)	Standard JSON-RPC 2.0 code for a structurally invalid request object.
JSONRPC_METHOD_NOT_FOUND (−32601)	Standard JSON-RPC 2.0 code returned when the requested method name is not registered.
JSONRPC_INVALID_PARAMS (−32602)	Standard JSON-RPC 2.0 code for missing, wrong-type, or out-of-range parameters; used extensively by all handlers.
JSONRPC_INTERNAL_ERROR (−32603)	Standard JSON-RPC 2.0 internal error code; also used for the select case default branch in arithmetic op helpers to flag an invalid op-code.
JSONRPC_SERVER_ERROR (−32000)	Server-defined error for general server-side failures (pipe errors, oversized requests, benchmark abort).
JSONRPC_MATRIX_DIMENSION_MISMATCH (−32001)	Server-defined error returned by ValidateMatrixParams when rows or columns is not 4.
JSONRPC_INVALID_DATA_TYPE (−32002)	Server-defined error returned by ValidateMatrixParams when the datatype field does not match the expected type for the called method.
JSONRPC_OVERFLOW_ERROR (−32098)	Server-defined error returned by PerformArithmeticOp when the 64-bit result of an integer operation exceeds the 32-bit signed range.
JSONRPC_DIVISION_BY_ZERO (−32099)	Server-defined error returned by DivideInt_handler, DivideReal_handler, and DivideComplex_handler when the divisor is zero.
OP_ADD=1, OP_SUBTRACT=2, OP_MULTIPLY=3	Operation selector codes passed to PerformArithmeticOp, PerformRealArithmeticOp, and PerformComplexArithmeticOp to choose the arithmetic operation without code duplication.
INT_MAX=2147483647, INT_MIN=−2147483648	32-bit signed integer bounds used by PerformArithmeticOp to detect overflow after computing in 64-bit.

FILENAME: ServerComm.f90 — module ServerComm (reference only — compiled into static library)

Name	Summary
module ServerComm	Core server communication module. Not compiled directly into the server program — compiled into JSONRPCStaticLibraryRevA.lib; its interface is provided via servercomm.mod in AdditionalDirectories. This source copy is kept for reference only.
GetTimeStamp	Returns the current time as an HH:MM:SS formatted 8-character string using date_and_time. Used as a prefix on all console output lines in ProcessMessages.
SendErrorResponse	Builds and sends a JSON-RPC error response object using string concatenation (without json-fortran), frames it with Content-Length, and writes it to the pipe. Optionally includes a "data" field for additional context.
SendProgressNotification	Sends a progress update notification with wire format {"jsonrpc":"2.0","method":"progress","params":{"percent":N}}. Uses object params — distinct from SendProgressToClient in Functions.f90 which uses array params [N] as required by the VB.NET client.
escape_simple	Performs minimal JSON string escaping on a message string, replacing ", \, newline, carriage return, and tab with their JSON escape sequences. Used by SendProgressNotification for the optional message field.
ProcessMessages	Main receive-dispatch loop. Reads Content-Length-framed messages from the named pipe, extracts the JSON body, classifies each as a request or notification via json_has_id_key, dispatches via jsonrpc_parse_message, and writes the response back. Exits on broken pipe or zero-byte read.
json_has_id_key	Scans a JSON string character by character for the four-character sequence "id" followed by optional whitespace and a colon. A bare substring search is unreliable because id can appear inside string values; requiring the colon confirms it is a JSON key occurrence.

LIST OF METHODS IN FORTRAN SERVER

Method Name	Summary
`addint`	Adds two 32-bit integers supplied as positional params. Uses a 64-bit intermediate to detect overflow before truncating to the 32-bit result; returns `JSONRPC_OVERFLOW_ERROR` if the result exceeds the signed 32-bit range.
`subtractint`	Subtracts the second integer from the first. Uses the same 64-bit overflow detection as `addint`.
`multiplyint`	Multiplies two 32-bit integers. Uses the same 64-bit overflow detection as `addint` and `subtractint`.
`divideint`	Divides the first integer by the second. Performs an explicit zero-check before dividing; returns `JSONRPC_DIVISION_BY_ZERO` if the divisor is zero.
`addreal`	Adds two double-precision (`REAL(8)`) values supplied as positional params. Result is rounded to 2 decimal places to avoid floating-point display artefacts.
`subtractreal`	Subtracts the second real number from the first. Result is rounded to 2 decimal places.
`multiplyreal`	Multiplies two double-precision real values. Result is rounded to 2 decimal places.
`dividereal`	Divides the first real number by the second. Returns `JSONRPC_DIVISION_BY_ZERO` if the divisor is zero; result rounded to 2 decimal places.
`addcomplex`	Adds two complex numbers supplied as strings in `"re±imi"` format (no brackets). Parses each string into real and imaginary parts, performs the addition, and returns the result as a formatted complex string.
`subtractcomplex`	Subtracts the second complex number from the first. Same string parse/format approach as `addcomplex`.
`multiplycomplex`	Multiplies two complex numbers using standard complex multiplication rules `(ac−bd)+(ad+bc)i`. Same string parse/format approach as `addcomplex`.
`dividecomplex`	Divides the first complex number by the second using the conjugate-of-denominator formula. Returns `JSONRPC_DIVISION_BY_ZERO` if $re\_b^2+im\_b^2=0$.
`sendmessage`	Echoes back the first string parameter as the result. Validates that params is a non-empty array and that the first element is a non-empty string.
`namedparameters`	Accepts a JSON object with any number of named fields, prints each key-value-type triple to the server console, and returns a string indicating the total count of named parameters received.
`mandelbrotbenchmark`	Runs a Mandelbrot set computation over a $(2000\times seed)^2$ grid as a CPU benchmark. Accepts a seed value (1–10), sends 10% progress milestones as notifications during the run, supports stop/pause/resume signals from the client mid-computation, and returns a floating-point checksum when complete.
`matrixintegertranspose`	Accepts a 4×4 integer matrix as a row-major JSON array, transposes it using Fortran's intrinsic `TRANSPOSE()`, and returns the result in row-major order.
`matrixintegercopy`	Accepts a 4×4 integer matrix and returns it unchanged. Functions as a round-trip identity operation for verifying correct serialisation and deserialisation.

Method Name	Summary
`matrixintegersquare`	Accepts a 4×4 integer matrix, validates each element is within `[-100, +100]` to prevent 32-bit overflow (since 100^2=10000), squares each element individually, and returns the result matrix.
`matrixrealcopy`	Accepts a 4×4 `REAL(8)` matrix, validates each element is within `[-100.0, +100.0]` (matching the VB.NET client's signed spinner range), and returns the values unchanged.
`matrixrealtranspose`	Accepts a 4×4 `REAL(8)` matrix, validates each element is within `[0.0, +100.0]` (matching the VB.NET non-negative spinner), transposes it using `TRANSPOSE()`, and returns the result in row-major order.
`matrixrealsquare`	Accepts a 4×4 `REAL(8)` matrix with no range validation (REAL(8) has sufficient range), squares each element, and returns the result matrix.
`matrixtextcopy`	Accepts a 4×4 matrix of string values (up to 6 characters each) and returns them unchanged. Uses a `block` construct for safe allocatable-to-fixed-length string assignment.
`matrixtexttranspose`	Accepts a 4×4 matrix of strings, transposes it using `TRANSPOSE()`, and returns the result in row-major order.
`matrixlogicalcopy`	Accepts a 4×4 boolean matrix (VB.NET `True`/`False` converted automatically by json-fortran to Fortran `.TRUE./.FALSE.`), and returns each value unchanged as the lowercase string `"true"` or `"false"`.
`matrixlogicaltranspose`	Accepts a 4×4 boolean matrix, transposes it using `TRANSPOSE()`, and returns the result as lowercase `"true"`/`"false"` strings in row-major order.
`matrixcomplexcopy`	Accepts a 4×4 matrix of complex numbers as bracketed strings `"(re±imi)"`, parses each into `COMPLEX(8)`, and returns them reformatted as a round-trip identity operation.
`matrixcomplextranspose`	Accepts a 4×4 matrix of bracketed complex strings, parses them into a `COMPLEX(8)` array, transposes using `TRANSPOSE()`, and returns the result as formatted strings in row-major order.
`matrixcomplexsquare`	Accepts a 4×4 matrix of bracketed complex strings, parses each into `COMPLEX(8)`, squares each using Fortran complex arithmetic (`c*c`), and returns the results as formatted strings.

FORTRAN EXTERNAL FILES

External File	Summary
JSONRPCStaticLibraryRevA.lib	The primary static library providing the JSON-RPC 2.0 protocol engine. Contains the compiled object code for jsonrpc_init, jsonrpc_register_method, jsonrpc_shutdown, jsonrpc_parse_message, and ProcessMessages. Linked explicitly via AdditionalDependencies in the .vfproj for both Debug|x64 and Release|x64 configurations.
json-fortran.lib	The json-fortran open-source library providing all JSON parsing and construction routines (json_core, json_value, json_module). Used throughout every handler file to read params and build result objects. Listed under Resource Files in the .vfproj; the linker picks it up from the project directory.
jsonrpc_protocol.mod	Compiler interface module for the jsonrpc_protocol module compiled into JSONRPCStaticLibraryRevA.lib. Exposes jsonrpc_init, jsonrpc_register_method, jsonrpc_parse_message, and jsonrpc_shutdown to the main program. Located in .\AdditionalDirectories, referenced via AdditionalIncludeDirectories.
jsonrpc_types.mod	Compiler interface module exposing the method_entry derived type and method_handler_interface abstract interface from the static library. Required explicitly in the main program because Intel ifx does not re-export derived types through a use-association chain.
jsonrpc_helpers.mod	Compiler interface module for internal helper routines compiled into the static library. Provides support utilities used by the protocol dispatch layer (parameter extraction helpers, response builders).
jsonrpc_server_state.mod	Compiler interface module exposing the server's shared state (including g_hPipe, the global pipe handle written by WriteJsonNotification on disconnect). Consumed internally by the static library's dispatch and communication routines.
jsonrpc_utils.mod	Compiler interface module for general-purpose utility routines within the static library, such as string and buffer helpers used by the protocol layer.
jsonrpc_validation.mod	Compiler interface module for JSON-RPC request validation routines compiled into the static library — verifies jsonrpc version field, presence of method, and structural correctness before dispatch.
servercomm.mod	Compiler interface module for the ServerComm module compiled into the static library. Exposes ProcessMessages, SendErrorResponse, SendProgressNotification, GetTimeStamp, escape_simple, and json_has_id_key to the server program.
json_module.mod	Primary compiler interface module for the json-fortran library. Exposes json_core and json_value — the two central types used in every handler subroutine signature for parsing params and building results.

External File	Summary
`json_file_module.mod`	Compiler interface module for json-fortran's file I/O layer (`json_file` type). Included as part of the json-fortran build; not directly used by the server's handler code, but required for a complete module resolution of `json_module`.
`json_value_module.mod`	Compiler interface module for json-fortran's `json_value` derived type and its associated procedures. Provides the underlying pointer-based tree structure that `json_core` operates on.
`json_kinds.mod`	Compiler interface module defining json-fortran's kind parameters (integer and real precision constants). Required for consistent type compatibility between json-fortran internals and the server's `REAL(8)` and `INTEGER` declarations.
`json_parameters.mod`	Compiler interface module for json-fortran compile-time configuration constants such as default real kind, string length limits, and CK/IK/RK kind aliases.
`json_string_utilities.mod`	Compiler interface module for json-fortran's internal string manipulation routines (whitespace trimming, escape handling, numeric-to-string conversion). Resolved as a dependency of `json_module` during compilation.

Notes:

- All `.mod` files in `.\AdditionalDirectories` are pre-compiled interface files shipped with the static library or json-fortran build — they are consumed only by the compiler (`ifx`) and produce no linker output of their own.
- Intel Fortran built-in modules `ifwin` and `ifwinty` (Win32 API bindings used in `ConsoleUtils.f90` and the main program) are supplied by the Intel Fortran compiler installation itself and do not appear as separate files in the project directory.